

Python Docstrings: Technical Reference Handout

What is a Docstring?

A **docstring** (documentation string) is a literal string used to document a Python module, class, function, or method. Unlike comments (`#`), **docstrings are not ignored by the Python interpreter**; they are stored in the object's `__doc__` attribute and can be accessed programmatically.

Docstrings are enclosed in **triple quotes**: `"""` or `'''`. Triple double quotes (`"""`) are preferred per PEP 257.

Why Use Docstrings Instead of Comments?

Feature	Docstring	Comment
Interpreter Accessibility	Yes (obj.__doc__)	No (ignored at runtime)
Tool Integration	Yes (e.g., IDEs, help(), pydoc)	No
PEP Compliance	Yes (PEP 257)	No
Multi-line Capability	Yes	Limited (requires multiple #)

Docstring Syntax Examples

Function Docstring

```
def calculate_area(width, height):  
    """  
  
    Calculate the area of a rectangle.  
  
    Parameters:  
    width (float): The width of the rectangle.  
    height (float): The height of the rectangle.
```

Returns:

float: Area calculated as width * height.

"""

return width * height

Class Docstring

class Vector:

"""

Represents a 2D mathematical vector.

Attributes:

x (float): X-coordinate.

y (float): Y-coordinate.

"""

def __init__(self, x, y):

"""Initialize a vector with x and y components."""

self.x = x

self.y = y

Module-Level Docstring

"""

geometry_utils.py

Utility functions for geometric calculations including area, perimeter,
and volume for basic shapes.

"""

Docstring Formats

Python does not enforce a specific format, but the most widely accepted styles are:

1. PEP 257 (Python Standard)

Basic conventions like starting with a one-line summary, followed by an optional detailed description.

2. reStructuredText (reST)

Used by Sphinx for generating documentation.

```
def subtract(a, b):  
    """  
  
    Subtract one number from another.  
  
    :param a: The number to subtract from.  
    :param b: The number to subtract.  
    :returns: The result of a - b.  
    :rtype: float  
    """  
  
    return a - b
```

3. Google Style

```
def multiply(a, b):  
    """  
  
    Multiply two numbers.  
  
    Args:  
        a (int): First operand.  
        b (int): Second operand.
```

Returns:

int: The product of a and b.

"""

return a * b

4. NumPy Style

```
def divide(a, b):
```

"""

Divide two numbers.

Parameters

a : float

Dividend.

b : float

Divisor.

Returns

float

Result of division.

"""

return a / b

Accessing Docstrings at Runtime

```
>>> print(calculate_area.__doc__)
```

Calculate the area of a rectangle.

Parameters:

width (float): The width of the rectangle.

height (float): The height of the rectangle.

Returns:

float: Area calculated as width * height.

You can also use:

help(calculate_area)

Best Practices (PEP 257)

- Use triple double quotes `"""`, even for one-liners.
- Begin with a short summary line.
- Leave a blank line after the summary if more detail follows.
- Document parameters and return types explicitly.
- Be concise but complete.
- Avoid repetition of the function or class name in the docstring.
- Don't describe implementation details unless they are relevant to the user.

Common Mistakes to Avoid

Mistake	Why It's Wrong
Using single-line comments instead of docstrings	Not accessible via <code>.__doc__</code> ; not recognized by tools
Omitting triple quotes	Not a valid docstring; becomes a regular string or syntax error

Mistake

Why It's Wrong

Not updating docstrings

Leads to misleading or outdated documentation

Putting docstrings in the wrong place

Must immediately follow the def, class, or module line

Summary

- Docstrings are string literals **not ignored** by Python — they live in memory and provide introspectable documentation.
 - Follow **PEP 257** conventions and adopt a consistent formatting style (Google, NumPy, reST).
 - Docstrings improve **readability, maintainability**, and **tool support**.
-